Lexical Task Heads beyond Simple Tasks: Logit Lens and Jacobian Lens Readouts

Filippo Merlo

Draft — October 27, 2026

Abstract

Lexical task heads are attention heads whose output, decoded into the vocabulary, names the task the prompt asks for (Yang et al., 2026). They were identified with the per-head logit lens on simple single-step tasks, such as mapping a country to its capital. We ask whether the same method finds them for complex tasks defined by explicit rules (Fu et al., 2026), which need several inference steps, and whether Head-JLens (Merlo, 2026), which decodes each head's output after mapping it through a Jacobian of the remaining layers averaged over many texts, finds lexical task heads that the logit lens misses. In Llama-3.1-8B-Instruct, we identify lexical task heads under both readouts on 14 simple tasks and 5 complex tasks at three difficulty levels, with rule-based and example-based prompts, and test each head set by activation patching.

1 Introduction

A language model's accuracy on a task can change a great deal with the wording of the prompt that asks for it: on Llama-3.1-8B-Instruct, prompts for the same antonym items differ in accuracy by up to a factor of four (Yang et al., 2026, Fig. 1). Yang et al. (2026) explain part of this variation with *lexical task heads*, attention heads whose outputs, decoded into the vocabulary with the logit lens, name the task ("opposite" for antonyms, "cities" for capitals). The heads are largely shared between instruction-based and example-based prompts, patching their mean activation into a prompt without task information raises task accuracy, and prompts that fail activate them less.

Two limits of that work motivate this project. First, its 17 tasks are simple single-step mappings (country to capital, word to antonym), and the authors note that "it remains unclear if these shared lexical mechanisms hold for more abstract tasks or tasks that are less easily defined by simple vocabulary clusters" (Yang et al., 2026, Sec. 4). Fu et al. (2026) provide such tasks: five novel tasks defined by explicit rules (a card game, a board game, a composed arithmetic operator, noun-class agreement in an artificial language, and lexical-category inference), each at three difficulty levels and each given as rules, as examples or as both. Second, the logit lens decodes a head's output directly and leaves out the computation of the layers that follow it; Yang et al. (2026, Sec. 4) note that task representations "encoded in a non-lexical, abstract format" may not project cleanly to tokens. Head-JLens reads each head through a Jacobian map of the remaining layers, and on Llama-3.1-8B-Instruct it ranks annotated intermediate words, which the prompt never states (such as the bridge entity of a two-hop question), higher than the per-head logit lens at most layers on five of six evaluation sets (Merlo, 2026, Sec. 3).

We therefore ask whether Yang et al.'s method identifies lexical task heads for the complex tasks of Fu et al. (2026), and how the logit lens and Head-JLens compare as readouts for identifying them.

Contributions.

1. A test of the lexical-task-head method on complex, rule-defined tasks at three difficulty levels, with a causal check of the identified heads.

2. A comparison of the per-head logit lens and Head-JLens for identifying lexical task heads, on simple and complex tasks.

2 Related work

Task representations across prompting styles. Function vectors are sums of the mean outputs of a few attention heads that, added to the residual stream, trigger a task in another context (Todd et al., 2024). Davidson et al. (2025) extract them from instructions as well as from demonstrations and find that the two prompting styles engage mostly different heads: of the 20 top heads in each model, 2 to 7 are shared. Yang et al. (2026) identify a different kind of task representation, lexical task heads, whose outputs decode to words describing the task; 73% of them are shared between instruction-based and example-based prompts on average, and they are largely disjoint from function-vector heads (Yang et al., 2026, App. I). We test the lexical-task-head method on complex tasks.

Reading attention heads in the vocabulary. The logit lens decodes an intermediate state with the model's final normalization and unembedding (nostalgebraist, 2020); applied to one head's output, it is the readout of Yang et al. (2026). The Jacobian lens of Gurnee et al. (2026) instead maps a residual-stream state through a Jacobian of the remaining layers, averaged over many texts. Head-JLens moves this map to the concatenated head outputs, so that each head is read through the layers that follow it; in Llama-3.1-8B-Instruct it ranks annotated intermediate words higher than the per-head logit lens at most layers on five of six evaluation sets, and heads selected with it are better sites for edits (Merlo, 2026). We compare the two readouts on a different use: identifying task-descriptive heads.

Learning tasks from rules and from examples. Fu et al. (2026) compare learning from rules, from examples and from both on five novel tasks. Rules outperform examples overall, adding examples to rules gives no significant gain, and the advantage of rules is largest for tasks that need algebraic computation. Their study is behavioural; we use their tasks to study task representations.

3 Research questions

RQ1 Does the method of Yang et al. (2026) identify lexical task heads for the complex tasks of Fu et al. (2026) under rule-based and example-based prompts, and do the identified heads have a causal effect on task accuracy?

RQ2 How do the per-head logit lens and Head-JLens compare as readouts for identifying lexical task heads, on simple and on complex tasks?

RQ3 Are the lexical task heads of a complex task shared between rule-based and example-based prompts, as Yang et al. (2026) find for simple tasks?

Hypothesis 1 (replication)**.** *On the simple tasks, Logit-head reproduces Yang et al. (2026): lexical task heads are found for every task, and patching them into prompts without task information raises accuracy above the unpatched baseline.* Falsified *if, on the same model, the method finds no heads or no accuracy gain for tasks where Yang et al. (2026) report both.*

Hypothesis 2 (complex tasks)**.** *Following the limitation stated by Yang et al. (2026, Sec. 4), the complex tasks yield fewer lexical task heads per task and a smaller causal effect than the simple tasks under Logit-head.* Falsified *if they yield as many heads with as large an effect.*

Hypothesis 3 (readouts)**.** *Following Merlo (2026), J-head identifies at least the lexical task heads that Logit-head identifies, and the heads found only by J-head have a causal effect above baseline.* Falsified *if the J-head set misses heads of the Logit-head set, or if the heads found only by J-head have no causal effect.*

Scope. We do not explain behavioural variation across wordings (the failure analyses of Yang et al., 2026, Sec. 2.4), and we use one model.

4 Framework

Tasks and prompts. A task t is given by a dataset D_t of inputs and correct outputs. For the complex tasks, each difficulty level (and, for Lexical Category Inference, each logic variant) is a separate task. Each prompt for t has a style $s \in \{\text{rule}, \text{example}, \text{combined}\}$: its task information is a rule text, a set of demonstrations, or both, followed by the input x. $P_{t,s}$ is the set of prompts of style s for t, and $P^{+}_{t,s} \subseteq P_{t,s}$ those the model answers correctly and straight away, with the answer as the first generated text (in reasoning mode, Sec. 5.3: correctly after reasoning).

Heads and readouts. Following Merlo (2026, Sec. 2), $z_{l,h}(p)$ is the output of head h in layer l at the last position of p, before the output projection, and $c_{l,h}(p) = W^{(h)}_{O,l} z_{l,h}(p)$ its write to the residual stream. With $q_v = g \odot w_v$ the gain-scaled unembedding row of token v, the two readouts score v by

$$\ell^{\mathrm{L}}_{l,h}(p)[v] = q_v^{\top} c_{l,h}(p) \quad \text{(Logit-head)}, \qquad \ell^{\mathrm{J}}_{l,h}(p)[v] = q_v^{\top} J_{l,h}\, z_{l,h}(p) \quad \text{(J-head)}.$$

$T^{R}_{k,l,h}(p)$ is the set of the k highest-scoring tokens under readout $R \in \{\mathrm{L}, \mathrm{J}\}$, and V_t the term list of task t.

Definition 1 (Lexical task head)**.** *Following Yang et al. (2026, Sec. 2.2.2), head (l,h) is a lexical task head for t on prompt p under readout R if $|T^{R}_{k,l,h}(p) \cap V_t| \geq n$. It is a lexical task head for t and style s if this holds for at least a fraction ρ of the prompts in $P^{+}_{t,s}$. We use $n = 1$, $k = 10$, $\rho = 0.1$, as Yang et al. (2026). $H^{R}_{t,s}$ denotes the set of such heads.*

5 Methods

5.1 Model

Llama-3.1-8B-Instruct: it is the main model of Yang et al. (2026), and the only model with released Head-JLens matrices.

5.2 Tasks

Simple tasks. The 14 tasks used by both Yang et al. (2026) and Davidson et al. (2025), with the datasets of Todd et al. (2024): antonym, synonym, singular–plural, present–past, country–capital, country–currency, landmark–country, park–country, product–company, next-item, prev-item, English–French, English–German, English–Spanish. Rule-based prompts: the six instruction templates per task of Yang et al. (2026, App. A.3.1). Example-based prompts: Yang et al. (2026, App. A.3).

Complex tasks. The five tasks of Fu et al. (2026), each at three difficulty levels (Lexical Category Inference also in AND and OR variants): Set Game, Tapatan, Operator Function, Noun Class Agreement, Lexical Category Inference. Data from their public release. Rule-based prompts: their rules-only prompts (Fu et al., 2026, App. F) and further wordings (Sec. 5.3). Example-based and combined prompts: their minimum-coverage demonstration sets (Fu et al., 2026, Sec. 2.5).

Why these tasks. Lexical task heads and their term lists were defined on the simple tasks (Yang et al., 2026), so the method applies unchanged and our results can be compared with theirs. Yang et al. (2026, Sec. 4) restrict their study to tasks with simple lexical descriptions. The complex tasks are novel, need several inference steps, and are given in rule, example and combined forms that specify the same task on the same test items, so the task is held fixed while the way it is communicated varies (Fu et al., 2026, Sec. 1–2). Their difficulty levels change the number of variables and inference steps (Fu et al., 2026, Table 1).

Answer format. As in the source studies: the templates of Yang et al. (2026) for the simple tasks; the answer formats and generation limits of Fu et al. (2026, App. A) for the complex tasks. This keeps accuracy comparable with the reported results. Fu et al.'s instruction-tuned models often explain before answering, so forcing a direct answer would be a different setting. Lexical task heads are read at the last prompt token, where the answer is produced only if the model answers straight away; prompts on which it explains first are therefore left out of $P^{+}_{t,s}$ (Sec. 4), except for the tasks run in reasoning mode (Sec. 5.3).

5.3 Task-representation methods

Lexical task heads (Yang et al., 2026, Sec. 2.2.2). Each attention head's output at the last prompt token is decoded with the two readouts of Sec. 4: Logit-head, the per-head logit lens used by Yang et al. (2026) (nostalgebraist, 2020), and J-head (Head-JLens), which applies the head's columns $J_{l,h}$ of the per-layer Jacobian map fitted on WikiText-103 (Merlo, 2026, Sec. 2). Fitted matrices for Llama-3.1-8B-Instruct are released with Merlo (2026, Sec. S4). Under each readout, lexical task heads are identified with Definition 1. Term lists: Yang et al. (2026, App. A.2.1) for the simple tasks; for the complex tasks, see below. Causal check, as in Yang et al. (2026, Sec. 2.3.2, App. G): the mean of $c_{l,h}$ over $P^{+}_{t,s}$, multiplied by a scaling factor α, replaces the output of every head in $H^{R}_{t,s}$ at the last position of a prompt without task information, and accuracy is measured for α from -10 to 10. The control patches mean activations from a different task at the same heads.

What we add to the complex tasks. Fu et al. (2026) provide the data, one rules-only prompt per task and difficulty level, and the example-based and combined prompts. To apply lexical task heads to them we add three things.

1. *Rule wordings*: at least six per original rule prompt (App. A), each stating exactly the same rules. They are generated with GPT-5.5 through ChatGPT from a paraphrase prompt that keeps placeholders, output labels and the answer-format instruction verbatim (App. B), and each is checked by us against the original; App. C lists them. The lexical-task-head criterion for a prompt style counts heads over at least 10% of that style's prompts; Yang et al. (2026) use six per task. The complex rules are longer and can be phrased in more ways, hence at least six.

2. *Term lists* of task-descriptive words, of the kind in Yang et al. (2026, App. A.2.1): single words that describe the task rather than its answers, with case and number variants. They are generated with GPT-5.5 through ChatGPT from the rule prompt (App. B), as Yang

et al. (2026, App. O) generated term lists with LLMs, and checked by us. Each word is kept in its bare and space-prefixed forms that are single tokens of the model's tokenizer. There is one list per original rule prompt (App. A): Tapatan's levels share one prompt and change only board size, piece count and line length, whereas the other tasks add rules or, for Operator Function, change the function (Fu et al., 2026, Table 1, App. F). Lists are task-specific, not item-specific: for Lexical Category Inference, whose category definitions change from item to item, the list contains only words that describe the task itself. App. D lists them.

3. *Prompts without task information* for the causal check: the test input alone, without rules or examples, the counterpart of the zero-shot prompts of Yang et al. (2026, Sec. 2.3.2).

Reasoning mode. For the complex tasks on which E0 finds that the model does not answer straight away (so far Set Game and Operator Function), we follow the instruction-tuned setup of Fu et al. (2026, App. A) and let the model reason: chat template, greedy decoding, and an output limit of 6,000 new tokens, their limit for Operator Function, for all of these tasks. Answers are extracted as in their released code: after whitespace is collapsed, the first `\boxed{}`, read as three cards of the form `(animal | biome | ...)` for Set Game and as an integer for Operator Function. Their code has no parser for Tapatan; we take the first of `A win`, `B win`, `continue` in the output. An output without an extractable answer within the limit is wrong. $P^{+}_{t,s}$ then contains the prompts whose final answer is correct, whatever precedes it. Definition 1 is applied at two positions: the last prompt token, as for the other tasks, where heads show the task as identified from the prompt; and the pre-answer position, the token after which the first answer token is written, found by one forward pass over the prompt and the model's own output, where heads show the task as represented when the answer is produced. At the pre-answer position, the copy share also checks the generated text. In the causal check, each head set's scaled mean activation, taken at the position where the set was found, is patched at the last prompt position of a prompt without task information and kept at every generated position, and the final answer is scored. Heads at the last prompt token are compared across all tasks; heads at the pre-answer position and the accuracies after reasoning are reported separately. To bound the cost of long generations, these tasks use a fixed sample of items per task, style and wording and a coarser grid of scaling factors [TBD: sample size and grid].

Why these readouts. Lexical task heads locate a task representation and give its content in words (Yang et al., 2026). Logit-head reproduces their readout. J-head reads each head through the layers that follow it: in Llama-3.1-8B-Instruct it ranks unstated intermediate words higher than Logit-head at most layers on five of six evaluation sets, and its readouts are better guides for edits (Merlo, 2026, Secs. 3–4). A lexical task head's decoded task word is likewise not the answer, and Yang et al. (2026, Sec. 4) note that their logit-lens criterion may miss task representations that do not project cleanly to tokens.

5.4 Measures

For each task t, style s and readout R:

- *Accuracy* of each prompt, with the evaluation of the source paper: that of Yang et al. (2026) for the simple tasks and that of Fu et al. (2026) for the complex tasks.
- *Head count* $|H^{R}_{t,s}|$ and the distribution of the heads over layers.
- *Overlap* between two head sets A and B: $|A \cap B|/|A|$ and $|A \cap B|/|B|$. Compared pairs: $H^{\mathrm{L}}_{t,s}$ and $H^{\mathrm{J}}_{t,s}$ (RQ2); $H^{R}_{t,\mathrm{rule}}$ and $H^{R}_{t,\mathrm{example}}$ (RQ3).

- *Causal effect*: the highest accuracy over α minus the unpatched accuracy, for the head set and for the control.
- *Copy share*: the share of prompts on which a head's matched terms ($T^R_{k,l,h}(p) \cap V_t$) occur in the prompt text, averaged over the heads of $H^R_{t,s}$. It separates heads that may copy task words from the prompt from heads that do not.

6 Experiments

6.1 E0: Behavioural baseline

Question: on which tasks and difficulty levels does the model perform above chance? *Procedure*: accuracy of every wording of every style, on all tasks, and the share of prompts on which the model does not answer straight away. *Output*: the pairs of task and style used in E1 and E2, those whose accuracy exceeds chance, and the prompts $P^+_{t,s}$. The share of prompts left out decides whether other ways of handling them are needed.

6.2 E1: Simple tasks

Question: H1, and RQ2 on the simple tasks. *Procedure*: identify $H^{\mathrm{L}}_{t,s}$ and $H^{\mathrm{J}}_{t,s}$ for the 14 simple tasks, for rule-based and example-based prompts (Yang's six instruction templates; 1, 5, 10, 20 and 30 shots); compute the measures of Sec. 5.4; run the causal check on prompts without task information. *Decides*: whether Logit-head reproduces Yang et al. (2026) on our setup, and how J-head differs from it where the method is known to work.

6.3 E2: Complex tasks

Question: RQ1, H2, RQ2 on the complex tasks, and RQ3. *Procedure*: as in E1, for each complex task kept by E0, with the added wordings and term lists (Sec. 5.3); example-based and combined prompts from Fu et al. (2026). *Decides*: whether lexical task heads are found and causally effective for complex tasks, at which difficulty levels, under which readout, and whether they are shared with example-based prompts.

7 Plan

Steps.

1. Build the additions of Sec. 5.3: rule wordings (each checked to state the same rules as the original prompt), term lists, prompts without task information.
2. Run E0 and select tasks.
3. Run E1.
4. Run E2.

Reused code and data: the lexical-task-head code of Yang et al. (2026); the Head-JLens code and matrices of Merlo (2026); the data and generation code of Fu et al. (2026); the task datasets of Todd et al. (2024).

Limitations. One model. The term lists are generated by an LLM and checked by hand, and the identified heads depend on them. Rule-based prompts contain task words, so a head may decode them by copying from the prompt; the copy share measures this but does not decide it. Under J-head, some sites rank tokens highly on unrelated prompts as well (Merlo, 2026, Sec. 3.6), so a larger J-head set is not by itself evidence of more task representation; the causal check is needed. Prompts on which the model explains before answering are left out, which may remove some tasks or styles from E1 and E2.

Deliverables. Head sets, measures and causal-check curves for every task, style and readout.

References

Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. Do different prompting methods yield a common task representation in language models? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.12075.

Xiang Fu, Seungmin Cho, Yukyung Lee, and Najoung Kim. LLMs learn better in-context from rules than from examples. *arXiv preprint arXiv:2609.03213*, 2026.

Wes Gurnee, Nicholas Sofroniew, Adam Pearce, et al. Verbalizable representations form a global workspace in language models. Transformer Circuits Thread, 2026. URL https://transformer-circuits.pub/2026/workspace/index.html.

Filippo Merlo. Reading attention heads through the Jacobian lens. Manuscript, October 2026. https://github.com/filippo-merlo/head_level_jlens, 2026.

nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020. URL https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.15213.

Zhuonan Yang, Jacob Xiaochen Li, Francisco Piedrahita Velez, Eric Todd, David Bau, Michael L. Littman, Stephen H. Bach, and Ellie Pavlick. Shared lexical task representations explain behavioral variability in LLMs. In *International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2604.22027.

A Original rule prompts

The rules-only prompts of Fu et al. (2026), from their released code (https://github.com/tinlaboratory/rules-vs-examples, `rules-mode-prompts/`, MIT licence). Text in curly braces is filled per item. Tapatan uses the final-board-state prompt, whose levels differ only in its parameters: board size 3, 5, 7 and line length 3, 4, 5 for Easy, Medium, Hard (Fu et al., 2026, Table 1); the version in Fu et al. (2026, App. F) also states the maximum number of pieces per player and that at most one player has a winning line. The Medium and Hard prompts of Lexical Category Inference are identical within each logic variant.

Set Game

Easy.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has two attributes: an animal and a biome.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a set of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Medium.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has three attributes: an animal, a biome, and a food.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. There are 10 types of food: apple, potato, bread, cake, carrot, mutton, beef, cookie,
↪  pie, melon.
5. A GAME-SET is a set of three cards: For each attribute (animal, biome, food), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Hard.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has a number and three attributes: an animal, a biome, and a food.
2. Numbers range from 1 to 12.
3. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
```

```
4. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
5. There are 10 types of food: apple, potato, bread, cake, carrot, mutton, beef, cookie,
↪  pie, melon.
6. A GAME-SET is a set of three cards: For each attribute (animal, biome, food), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.
7. But only for the number, 2 of the cards should have the same number, and 1 of them should
↪  have a different number in order for the set to be valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Tapatan

all levels.

```
You will be learning a new board game. You will then be given one final board state from
↪  that game. Determine the current game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid of points with coordinates (x,y), where 0
↪  <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (x±1,y±1) when in bounds.
4. A player wins if they have {line_length} pieces in a straight line (horizontal, vertical,
↪  or diagonal) and each consecutive pair of points in that line is adjacent under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Operator Function

Easy.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function f is composed of two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z be inputs of the function.
3. To compute f(x,y,z), first multiply x and y.
4. Then, add z to the result from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Medium.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function g is composed of three arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z, a be inputs of the function.
3. To compute g(x,y,z,a), first subtract y from x.
4. Second, multiply the result from the previous operation by z.
5. Last, adds a to the result of the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Hard.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function h is composed of four arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z, a, b be inputs of the function.
3. To compute h(x,y,z,a,b), first add x and y.
4. Second, multiply the result from the previous operation by z.
5. Third, subtract a from the result of the previous operation.
6. Last, take the remainder when the result is divided by b.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Noun Class Agreement

Easy.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. Each noun phrase has the form DET NOUN.
3. The class A determiner is ka, and the class B determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences follow the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if, in each noun phrase, the determiner matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Medium.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 4 noun classes: A, B, C, D.
```

```
2. A noun phrase has the form DET ADJ-SUFF NOUN.
3. The class A determiner is ka, the class B determiner is ti, the class C determiner is su,
↪  and the class D determiner is vo.
4. Adjectives have the form STEM-SUFFIX. The suffix must match the noun's class.
5. The class A adjective suffix is -en, the class B adjective suffix is -os, the class C
↪  adjective suffix is -im, and the class D adjective suffix is -at.
6. Example adjective tokens are glim-en, prun-os, fen-im, zay-at.
7. The class A nouns are tav, fap. The class B nouns are bim, glarn. The class C nouns are
↪  sular, wug. The class D nouns are noko, zesh.
8. Sentences follow the template DET ADJ-SUFF NOUN VERB DET ADJ-SUFF NOUN.
9. Verbs are plain roots like dax, miv, lorp and never change.
10. A sentence is grammatical if and only if, in each noun phrase, both the determiner and
↪  the adjective suffix match the noun's class.
11. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Hard.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 6 noun classes: A, B, C, D, E, F.
2. A noun phrase has the form DET ADJ-SUFF NOUN.
3. The class A determiner is ka, the class B determiner is ti, the class C determiner is su,
↪  the class D determiner is vo, the class E determiner is ne, and the class F determiner
↪  is la.
4. Adjectives have the form STEM-SUFFIX. The suffix must match the noun's class.
5. The class A adjective suffix is -en, the class B adjective suffix is -os, the class C
↪  adjective suffix is -im, the class D adjective suffix is -at, the class E adjective
↪  suffix is -uk, and the class F adjective suffix is -esh.
6. The verb has the form PREFIX-ROOT-SUFFIX.
7. The prefix must match the class of the subject noun, and the suffix must match the class
↪  of the object noun.
8. The class A subject prefix is ge-, the class B subject prefix is du-, the class C subject
↪  prefix is ri-, the class D subject prefix is zo-, the class E subject prefix is pa-, and
↪  the class F subject prefix is li-.
9. The class A object suffix is -an, the class B object suffix is -eb, the class C object
↪  suffix is -ig, the class D object suffix is -ot, the class E object suffix is -ul, and
↪  the class F object suffix is -er.
10. Example verb tokens are ge-dax-eb, pa-miv-ig, du-lorp-er.
11. The class A nouns are tav, fap. The class B nouns are bim, glarn. The class C nouns are
↪  sular, wug. The class D nouns are noko, zesh. The class E nouns are tac, fep. The class
↪  F nouns are lurn, prax.
12. Sentences follow the template DET ADJ-SUFF NOUN PREFIX-ROOT-SUFFIX DET ADJ-SUFF NOUN.
13. Verb roots are plain roots like dax, miv, lorp. Only the verb's prefix and suffix change.
14. A sentence is grammatical if and only if the subject noun phrase's determiner and
↪  adjective suffix match the subject noun's class, the object noun phrase's determiner and
↪  adjective suffix match the object noun's class, the verb prefix matches the subject
↪  noun's class, and the verb suffix matches the object noun's class.
15. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Lexical Category Inference

Easy.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Medium and Hard, OR.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category lists multiple descriptions, an item is a member of that category if it
↪  satisfies at least one listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Medium and Hard, AND.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category lists multiple descriptions, an item is a member of that category only if
↪  it satisfies all listed descriptions.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}
```

```
List: {candidate_block}
Answer:
```

B Generation prompts

Both prompts are given to GPT-5.5 through ChatGPT; `{rule_prompt}` is an original prompt of App. A.

Rule wordings

```
You will be given a prompt that describes the rules of a task for a language model. Write
↪ {N} paraphrases of it.

Requirements:
1. Each paraphrase must state exactly the same rules as the original: the same constraints,
↪  conditions, quantities, names, symbols and labels. Do not add, remove, weaken or
↪  strengthen any rule, and do not add examples, hints or explanations.
2. Change the wording and the sentence structure. Reorder rules only where their order does
↪  not matter; if a rule refers to another rule by its number, keep the reference correct.
3. Copy verbatim every placeholder in curly braces (e.g. {board}), the answer-format
↪  instruction, and every output label or format string (e.g. "Yes or No", "A win, B win,
↪  or continue", \boxed{...}).
4. Keep unchanged the final lines that introduce the input and the answer (e.g. "Sentence:
↪  {sentence}" and "Answer:").
5. The paraphrases must differ from the original and from each other.

Return a JSON list of {N} strings.

Original prompt:
<<<
{rule_prompt}
>>>
```

Term lists

```
You will be given the rules of a task for a language model. List words that describe the
↪  task itself: what the task asks the model to do, its operation or relation, and its key
↪  concepts. For example, for the task "give the antonym of a word": opposite, reverse,
↪  antonym, contrary.

Requirements:
1. Single words only.
2. Do not list possible answers or output labels, words that only name the items of a
↪  particular input (e.g. specific animals, nouns or numbers), or placeholders.
3. Include singular and plural forms and capitalised forms. You may include translations
↪  into other languages.
4. Return a JSON list of strings.

Task rules:
<<<
{rule_prompt}
>>>
```

C Rule wordings

[TBD: generated wordings, after the check of Sec. 5.3; one subsection per original prompt of App. A]

D Term lists

Task	Level	Terms
Set Game	Easy	[TBD: list]
Set Game	Medium	[TBD: list]
Set Game	Hard	[TBD: list]
Tapatan	all	[TBD: list]
Operator Function	Easy	[TBD: list]
Operator Function	Medium	[TBD: list]
Operator Function	Hard	[TBD: list]
Noun Class Agreement	Easy	[TBD: list]
Noun Class Agreement	Medium	[TBD: list]
Noun Class Agreement	Hard	[TBD: list]
Lexical Category Inference	Easy	[TBD: list]
Lexical Category Inference	Medium, Hard (OR)	[TBD: list]
Lexical Category Inference	Medium, Hard (AND)	[TBD: list]

Decision log

2026-10-27 Scope: test the lexical-task-head method of Yang et al. (2026) on complex tasks and compare the per-head logit lens with Head-JLens. New title.

2026-10-27 Tasks: the 14 simple tasks shared by Yang et al. (2026) and Davidson et al. (2025); the five tasks of Fu et al. (2026).

2026-10-27 No main condition: rule-based and example-based prompts are treated alike (replacing the earlier choice of rules as main and examples as secondary).

2026-10-27 Method: lexical task heads, with Logit-head and J-head readouts. Function vectors, function-specific heads and the hidden-state difference between prompts are not used for now.

2026-10-27 At least six rule wordings per complex task and difficulty level.

2026-10-27 Term lists are task-specific, not item-specific; one per task, or per difficulty level where the rules change.

2026-10-27 Answer format as in the source studies.

2026-10-27 Model: Llama-3.1-8B-Instruct, the main model of Yang et al. (2026) and the only one with released Head-JLens matrices.

2026-10-27 Rule wordings and term lists for the complex tasks are generated with GPT-5.5 through ChatGPT and checked by hand.

2026-10-27 Accuracy is computed with each source paper's evaluation.

2026-10-27 Prompts on which the model does not answer straight away are left out; other ways of handling them are considered after E0.

2026-10-27 After an early E0 finding (no direct answers on Set Game and Operator Function), these tasks are run in reasoning mode (Sec. 5.3).